

against accidental and unintended modifications. Modifications could result in security breaches or conflicts. In other words, microgateways implemented as containers may help prevent breaches and conflicts because any security policy updates require the microgateway's redeployment. It is essential that the microgateway deployed for a microservice instance communicate with service registry and monitoring modules, allowing them to keep track of the operational status of the microservice that the microgateway is designed to protect.

When used with microservices, an API gateway or microgateway may serve to implement the following core features and functions:

- **Optimized end point:** The API gateway simplifies the process by providing aggregation, which results in fewer requests and responses. The API gateway can serve as a public interface for multiple individual APIs and their requests. Additionally, it performs the necessary protocol translation for client requests.
- **Circuit breaker:** Developers can set a threshold in the API gateway for microservice failed responses to requests and cut off requests when the failure is above the threshold. This avoids the possibility of a cascaded failure and allows management time to analyze logs, implement necessary fixes, and push necessary updates to address the failing instance of the microservice.
- **Load balancing:** When there are multiple instances of the same service, the load can be evenly distributed across the instances to avoid delayed responses or service crashes due to overload on any one instance.
- **Rate limiting (throttling):** The rate of requests going into a microservice should be limited to ensure continued availability of service for all clients.
- **Blue/green deployments:**¹⁰⁶ When a new version of a microservice is deployed, requests from customers using the old version can be redirected to the new version because the API gateway can be programmed to be aware of the locations of both versions.
- **Canary releases:** Only a limited amount of traffic is initially sent to a new version of a microservice to verify the accuracy of its response and performance. Once sufficient data demonstrate that the microservice is operating as designed, then the API gateway can direct the requests to the new version of the microservice.

IV.1.2 API Risk Mitigation

Mitigation strategies for common API risks include the following:¹⁰⁷

- **Inappropriate access controls:** API endpoints are vulnerable to unauthorized access as attackers can manipulate the specific object ID or other identifier that is sent with the request. Management should apply extra layers of security, such as for broken object-level

¹⁰⁶ This type of deployment provides an application release model when user traffic is progressively transferred from a prior version (i.e., old or blue) of an app or microservice to a nearly identical new (i.e., green) release. Both the blue and green versions are running in production at the same time.

¹⁰⁷ For more information, refer to "[OWASP API Security Project](#)."

authorization,¹⁰⁸ beyond those for standard end-point security, and implement appropriate authorization checks at the object level.

- **Inadequate user authentication:** Authentication processes are not functioning correctly or are inappropriate for the way the API is used (e.g., designed for internet of things [IoT] versus a web application) or without considering potential vectors of attack. Management should validate the user authentication process for the use of the API and consider MFA.¹⁰⁹
- **Security misconfiguration:** Security settings may be missing or inappropriate for the current expected functionality of the API, giving an attacker the opportunity to gain unauthorized access or knowledge of the system. Attackers may use automated tools to detect and exploit these flaws, leading to data or device compromise. Management should implement appropriate security, patch and update APIs, and disable unnecessary functions.
- **Excessive data exposure:** Developers try to implement APIs generically without addressing the sensitivity of the exposed data. Automated tools can have difficulty detecting this type of vulnerability because they cannot differentiate between legitimate data provided from the API and inappropriately provided sensitive data. This problem creates the potential for unauthorized data access. Developers should review the responses from the API to ensure that a request contains only legitimate data and to understand how the requestor will use the data. Developers should consider creating and implementing a scenario-based response process (including error codes) to validate that data returned by the APIs are appropriate for the scenario.
- **Lack of resources and data limiting:** API requests consume resources such as network, central processing unit (CPU), memory, and storage. When APIs do not implement rate limiting or limits are not set properly, it may lead to an API being unresponsive or unavailable (i.e., denial of service). Management should appropriately set all API processing parameters (e.g., execution timeouts, maximum allocatable memory, number of file descriptors, number of processes, request size, number of requests per client or resource, and number of records per page to return in a single request response). If even one of the limits is missing or set inappropriately, an API may be vulnerable.
- **Broken function level authorization:** Implementing proper authorization checks can be difficult because applications can contain many types of roles or groups and complex user hierarchy (e.g., subusers and users with more than one role). Because these checks are difficult to implement, what appears to be a legitimate API call or request may not be authorized, allowing unauthorized access to functionality, particularly administrative functions. Management should ensure that all access is denied by default and employ specific role-based permissions for users to gain access to functions. Appropriate personnel should review API end point security.
- **Mass assignment:**¹¹⁰ Users can update multiple object properties without appropriate authorization. The design of APIs provides for easier exploitation of mass assignment because they expose the underlying implementation of the application along with the

¹⁰⁸ Object-level authorization involves access-control tools applied at the code level to validate appropriate access to objects (e.g., fields such as names or values) by only authorized users.

¹⁰⁹ For more information, refer to FFIEC's [Authentication and Access to Financial Institution Services and Systems](#) on multifactor authentication.

¹¹⁰ The MITRE Corporation defines "[mass assignment](#)" as "a feature that allows simultaneous modification of multiple object attributes."

properties' names. Management should deny access to properties unnecessary for the user's role and only allow access when necessary.

- **Injection:** Client and external system data may not be validated and filtered appropriately, allowing malicious data to be introduced into systems and components via the API. Injection can lead to information disclosure, data loss, denial of service, or complete host takeover. Management should validate and filter all data coming to an API with appropriate parameters.
- **Improper asset management:** Management may not properly maintain API and related asset inventories or may not update and patch API security in a timely manner. This may either allow unauthorized access to sensitive data or compromise the server through older, unpatched API versions connected to the same database. Management should maintain current, accurate inventories of APIs and related assets, including services,¹¹¹ and regularly review them. Management should maintain logging of API activity (e.g., authentication, errors, redirects, rate limiting, and end points, including their parameters, requests, and responses). Management should consider security measures, such as API security firewalls, segregation of production and nonproduction data, and timely removal of older API versions.
- **Insufficient logging and monitoring:** Without sufficient, timely logging and monitoring, including API activity, it is difficult to identify patterns of potential malicious activity (e.g., threats and potential security weaknesses), which may lead to compromise of systems, components, and data. Management should perform continuous logging and monitoring of relevant API activity (e.g., failed authentication attempts, denied access, and input validation errors). Automated security information and event management tools with appropriately configured dashboards may help management monitor and manage API-related logging. To maintain confidentiality and integrity of sensitive data in activity logs, effective management secures logs and ensures only appropriate access to them.

With the volume and potential significance of API-related vulnerabilities, it is critical to incorporate planning for the exploitation of these vulnerabilities during API development and the entity's incident response plan. Neglecting to do so can result in the compromise of confidentiality, integrity, availability, and resilience as well as inadequate or untimely response during an incident. For more information on security and other topics related to APIs, refer to the *FFIEC IT Handbook's* "[Information Security](#)" and "[Architecture, Infrastructure, and Operations](#)" booklets.

IV.J Methodologies

A methodology¹¹² is used to facilitate the development, acquisition, or maintenance of a system or component project. Management should establish appropriate methodologies to enable effective management and control of system and component development, acquisition, and maintenance activities. Management can apply various methodologies, or a hybrid, to different projects based on stakeholder needs and project objectives. Methodologies can be a combination of strategies, design philosophies, and accepted practices designed as a structured, organized set

¹¹¹ For more information, refer to the *FFIEC IT Handbook's* "[Architecture, Infrastructure, and Operations](#)" booklet.

¹¹² *Merriam-Webster* defines "[methodology](#)" as "a particular procedure or set of procedures."

of rules that allow a project team to work together effectively. If the entity employs multiple methodologies, the chosen methodology should be defined for each IT project.

Effective management aligns implementation of any methodology with the overall strategic and business objectives. Planning, education, and communication are important elements to help ensure that critical resources are available when needed for IT projects. Management should promote effective planning through IT project management and support training for developers, quality management members, testers, and maintenance personnel. Entity staff should be proficient and qualified in the selected methodologies. Common examples used in the financial industry include waterfall and agile; however, examiners may encounter others.

IV.J.1 Waterfall

The waterfall methodology was the first widely used model. It is considered a traditional model for development, acquisition, and maintenance projects. This methodology provides a logical, rigid approach to managing a sequential series of tasks or steps in which each phase is completed before the next phase of the project can begin. This methodology may be useful for stable, less complex projects with well-defined objectives and requirements that are not prone to change. A primary benefit of this methodology is its ease of use and implementation. Additionally, because of the sequential progression of the project, coding issues are identified earlier, and project status is more easily measured than in other methodologies. However, it may be difficult or cost-prohibitive to return to a previous project phase to make corrections or changes in requirements requested by stakeholders. Other drawbacks include excessive time to produce a product, stakeholder difficulty in defining the desired end product, and difficulty making changes once started.

The waterfall methodology is a multiphase approach that begins with a requirements phase when stakeholders define the project requirements and most of the communication between stakeholders occurs. Once the first phase is completed, a product or service can then be designed, developed, tested, and implemented. This methodology allows only minimal stakeholder changes or iterations, and a final product is not provided until the implementation. Moreover, this methodology progresses through other phases until the final phase, which is the maintenance phase, during which any issues are corrected.

IV.J.2 Agile

An alternative to traditional project management methods (also known as predictive or waterfall management) is the use of adaptive or agile management methods. Incremental and iterative development principles form the foundation of agile methodologies.¹¹³ Such principles involve breaking down a project into smaller, manageable modules that can be designed, developed, vetted, and installed in repeated cycles allowing for additional features, adjustments, and corrections with each iteration. With incremental and iterative development, management does

¹¹³ The agile development methodologies represent a family of development types rather than a single methodology. Organizations exploring agile methodologies are working to define core values and guiding principles for enabling high-performing teams in implementing and executing with agility. These methodologies emphasize close communication and collaboration between the project team and stakeholders.

not need to identify all requirements at the beginning of the project, as changes may be added during later iterations. Agile projects are different from waterfall projects because they rely on (1) defining requirements before and during execution; (2) delivering frequent, iterative, and incremental products (i.e., not always fully completed); and (3) welcoming changes at any point in the process.¹¹⁴ Incremental and iterative development may be used for larger projects in which the final product's requirements are well-defined, but details may change or need enhancement during the process.

There are different models of incremental and iterative development (e.g., iterative, spiral,¹¹⁵ and prototyping modeling¹¹⁶), but they follow the main principle of a defined repetitive process. An agile methodology is an incremental and iterative development methodology that relies heavily on stakeholder interaction throughout the development process. During each phase, a deliverable is produced for the stakeholders, who provide feedback. The benefits of an agile methodology include earlier availability of a version of the deliverable and improved stakeholder satisfaction due to the regular feedback. A primary drawback of an agile methodology is the additional resource requirements (e.g., time, personnel, and funding) needed to implement the additional iterations.

Generally, an agile methodology involves dividing a project into smaller modules or “sprints.” Each sprint represents an independent project, with phases (e.g., planning, requirements, design, coding, testing, and documentation) similar to those of other development methodologies. In some cases, sprints may become part of a larger project. The project is evaluated at the end of each sprint, and adjustments are implemented as appropriate. An agile methodology is designed to anticipate change at regular intervals based on feedback from stakeholders, such as customers and business analysts, and derive business value from the feedback.

When applied effectively, the overall business goal of an agile methodology is to minimize risk of introducing defects, prevent cost overruns, and quickly resolve changes in scope before there is a significant impact to the project. An agile methodology aims to satisfy stakeholder requirements by providing rapid, continuous deliveries of useful systems and components, delivered in short intervals. It aims to release working elements of the systems and components at the end of each iteration, even if the overall product is incomplete. The feedback is used to make changes to the existing deliverable and the overall project plan.

Cross-functional teams representing various business lines are created to facilitate development. The diverse representation of these teams promotes consideration of multiple perspectives, improving visibility during the development process. The result of each iteration should be a

¹¹⁴ NIST Advanced Manufacturing Series 100-40, [Agile for Model-Based-Standards Development](#).

¹¹⁵ In the spiral model, the project iteratively goes through specific development phases (e.g., identify, design, build, and evaluate) with refinement in each iteration (or spiral) until development is complete. For more information, refer to National Aeronautics and Space Administration, [A Software Development Simulation Model of a Spiral Process](#).

¹¹⁶ Prototyping modeling involves developing a product with the look, feel, and limited functionality of the product under development. It can be used in conjunction with other models allowing developers to build prototypes during development. The prototype is built, tested, and adjusted until a working model is developed and can be produced as a final product or deployed throughout a system.

functional deliverable for the customer. Real-time feedback¹¹⁷ from customers or stakeholders is a key aspect of an agile methodology.

An agile methodology employs a flatter organizational structure to allow iterative delivery of systems or components. A common role supporting an agile methodology is the scrum master,¹¹⁸ which is similar to the traditional project manager role. However, unlike the project manager, a scrum master supports and serves the product owner and the development team by helping to remove obstacles to project deliverables.

An agile methodology may create unique challenges in documentation of development; therefore, the entity's documentation requirements should be clearly defined in the entity's project management requirements. Effective documentation relies on real-time communication between the development team and stakeholders, including end users. In any methodology, a lack of clarity or poor communication can negatively affect project deliverables and timelines (e.g., delays, scope creep, costs, and quality). This may have increased significance in an agile methodology because multiple parts of the overall project in process may be affected, leading to a greater impact than if only one segment of a project in process is affected.

Individual agile methodologies¹¹⁹ may differ in their specific processes; however, they are based on some common principles.¹²⁰ However, agile principles could be at odds with security principles. For example, the agile principle of speed to delivery could cause developers to leave out important security controls that would take longer to include but be invisible to the end user. To mitigate the risk of security flaws in an agile project, leaders can establish security standards that must be met before a solution is deployed for production use.

In prototyping models of systems and components (e.g., rapid application development), developers could bypass established change controls, leading to security and integrity risks. Following established control procedures when using these models is as important as when using any other development methodology type.

¹¹⁷ Feedback loops from customers and stakeholders may take several forms (e.g., emails, meetings) for communicating expectations and needed changes.

¹¹⁸ NIST defines a "[scrum master](#)" as "the role performing the management of the iteration and increments." A scrum is an agile project management approach that helps teams structure and manage their work through a set of values, principles, and practices toward a common goal.

¹¹⁹ Examples of agile methodologies include Extreme Programming, Scrum, Lean Software Development, Crystal Methodologies, Adaptive Software Development, Feature Driven Development, and Dynamic Systems Development Methodology.

¹²⁰ Common principles include short development iterations, minimal design upfront, emergent design and architecture, collective code ownership and ability for anyone to change any part of the code, direct communication and minimal or no documentation (i.e., the code is the documentation), and gradual building of test cases. Refer to Noopur Davis, [Secure Software Development Life Cycle Processes: A Technology Scouting Report](#), and related guidance, such as CISA, [Securing the Software Supply Chain: Recommended Practices Guide for Developers](#), and [Shifting the Balance of Cybersecurity Risk: Principles and Approaches for Security-by-Design and -Default](#).

IV.K Quality Management

Quality management refers to coordinated activities to direct and control an organization regarding quality. It helps management assess whether newly developed, procured, or modified systems and components are operating as envisioned, and are designed and comply with an entity's policies, standards, and procedures. Entities assess quality through manual, automated, or hybrid methods throughout the SDLC. Quality management should be an independent function from development, and prudent practices typically include implementing compensating controls when segregation of duties cannot be fully achieved. For example, developers should not test and validate systems and components that they developed.

Quality management comprises the functions of QA and quality control (QC), which together focus on providing confidence that quality requirements will be fulfilled. Although the goal is similar, QA and QC differ somewhat. QA refers to a planned, systematic pattern of all actions necessary to provide adequate confidence that a system, component, or facility conforms to established requirements. QC refers to the operational techniques and procedures (such as design analysis and inspection for defects) used to achieve quality requirements. QA and QC may be implemented separately or together, depending on the size and complexity of the entity and its development, acquisition, and maintenance activities.

Quality management is a critical part of well-managed development, acquisition, and maintenance activities. Comprehensive quality management, risk management, and testing standards provide a means to manage project risks and promote expected functionality, security, and interoperability in systems and components in a secure, resilient manner. Quality management policies, standards, and procedures should be applied to internally and externally developed programs and should address the following:

- **Commitment:** Successful projects generally include commitment from all involved parties. Senior management should adequately support and promote projects throughout an entity to promote success of a project. Failure by management to implement or support quality management programs hinders its ability to quickly detect project weaknesses and programming errors. Correcting weaknesses and errors detected late in a project life cycle is often more difficult and costly and may be less effective.
- **Expectations:** To improve quality management and minimize deficiencies, stakeholders should clearly define their expectations and effectively communicate their requirements initially and throughout a project. For example, expectations and requirements could include reliability, resilience, performance, simplicity, usability, interpretation of error messages, accessibility, interoperability, and robustness.
- **Completeness:** Each phase of a project life cycle should include procedures to follow and expected deliverables. Quality management personnel should verify the business case for a project, required functional features, and appropriateness of project considerations to determine completeness in each project phase before moving to the next phase. Audit and compliance personnel should verify that quality management includes quality standards to validate that the project meets internal and external requirements.
- **Scalability:** Projects vary in size and complexity. Quality management standards should match project characteristics and risks and may differ by project size and type. Quality

reviews may consider whether the entity's systems and components can scale, as necessary.

- **Measurability:** To accurately evaluate a project's success, management should assess results against defined expectations and requirements to determine project success. Quality management personnel should assess the quality of products and processes against measurable standards, metrics, and expectations.
- **Tracking:** Quality management personnel should validate that project personnel properly document a project's progress; record, report, and monitor problems to resolution; and communicate progress and concerns to management.
- **Independence:** Quality management personnel should be independent of the project they are reviewing to objectively assess elements, such as progress, problem resolution, documentation, and adequacy of the project.
- **Security:** Information security personnel perform a critical function in ensuring the confidentiality, integrity, availability, and resilience of internally developed and procured systems.¹²¹ Quality management personnel should validate the existence and effectiveness of information security considerations. The extent of security validation should be commensurate with risks associated with the project.
- **Legal and regulatory:** Legal and compliance personnel provide guidance throughout a project to ensure that it conforms with applicable statutory and regulatory requirements to mitigate project risks. Quality management personnel should assess statutory and regulatory requirements for deliverables (e.g., digital accessibility)¹²² and determine whether the systems and components can meet those requirements.

IV.L Documentation Standards

Thorough documentation allows stakeholders to clearly understand system and component functionality, security, control features, and resilience. Documentation conveys how a system or component was developed, how it functions, and where appropriate security and resilience points are included. Entity personnel can refer to documentation to effectively operate and maintain the systems and components. Additionally, well-prepared and maintained documentation reduces resilience risks, such as the loss of institutional knowledge common with personnel turnover. Documentation helps transfer key information (e.g., narratives, flowcharts, and any special system coding or file layouts) over time and across groups. Management should maintain documentation for both internally developed systems and components and externally procured products and services. Due to the sensitive nature of system and component documentation (e.g., internet protocol addresses, server names, and security controls), management should restrict access to sensitive documentation based on job description and need.

Examples of items that documentation may contain include feature and function descriptions, design specifications, programming descriptions, build procedures, and operating instructions. Also, documentation should be updated as systems and components are updated to ensure that the benefits described above continue.

¹²¹ For more information, refer to NIST SP 800-53, rev. 5, [Security and Privacy Controls for Information Systems and Organizations](#).

¹²² For more information, refer to [Section 508 of the Rehabilitation Act of 1973](#).

For acquired systems and components, management should obtain documentation from the third party and incorporate it into the entity's own stored documentation. Before purchase, effective management ensures (through an internal review or a third-party certification) that a procured system's documentation meets the entity's documentation needs and standards.

System documentation that includes current build and configuration instructions allows administrators to recover the system after a disruptive event. For a recovery process to be effective, these build and configuration instructions should be kept current as the system is maintained and changed. There are many build and configuration processes that are automated, and the scripts that automate those processes should be documented appropriately for future reference. However, in those cases it may be the manual action (unautomated) documentation that is at the highest risk of becoming stale.

Common documentation examples include the following:

- **System or component descriptions and topologies** that provide functional purposes, narrative explanations, and pictorial depictions of operating environments and the interrelated input, processing, and output functions of integrated systems and components.
- **System and program flowcharts and models** that identify the source and type of input information, processing, and control actions (automated and manual), and the nature and location of output information for a system or component. Additionally, program flowcharts present graphical views of the procedural program sequencing and provide a practical way to illustrate complex programs and routines. Administrators can use these flowcharts to view a system holistically or drill down to specific functions or interrelated components of a system or program.
- **System file and information layouts** provide specification useful in information collection, storage, and display.
- **Program documentation** details specific data input, processing, and output instructions, and should include documentation about system security. Examples of program documentation include program listings, source code, narrative comments, technical programming scripts,¹²³ and nontechnical descriptions of the scripts.
- **Manuals and procedures** inform IT personnel and end users on the operation of systems and components.

IV.M Post-Implementation Review

Organizations formally perform a post-implementation review (PIR) for system development and implementation projects to continuously improve. Processes and techniques that were effective are passed to future projects as effective practices. Problems and their solutions are also passed along to help future projects avoid pitfalls. In agile projects when there are multiple implementations, smaller reviews are completed after each task or deliverable is implemented. Typical PIR steps are

¹²³ Merriam-Webster defines a "[script](#)" in computing as "a sequence of instructions or commands for a computer to execute."

- Gathering data including project initiation documents, post-implementation metrics, and post-implementation stakeholder interview results.
- Analyzing any gaps between project initiation goals and objectives and actual outcomes, including the reasons for any gaps.
- Summarizing the analysis and any recommendations in a report to management.¹²⁴

Common project aspects analyzed in a PIR include the following:

- User satisfaction.
- Actual benefits compared to projected benefits.
- System or component defect level when implemented, which helps measure system testing effectiveness.
- Project management effectiveness.
- Actual costs and timeline versus business case projections.

When entities analyze project aspects like those above and pass the results to management and future project teams, they raise future project success probability and mitigate the risk of future project failure.

IV.N IT Project Management

Action Summary

Consistent use of appropriate project management policies, standards, and procedures can help personnel be effective in identifying and mitigating project risks before they materialize.

Examiners should review the following:

- Project plans, proposals, and status reports to determine the effectiveness of IT project manager collaboration with stakeholders (e.g., through the PMO) for oversight and completion of IT projects.
- Board and committee minutes to evaluate how the entity prioritizes projects relative to business goals and objectives, evaluates the project's effect on operations, and monitors and supports projects during execution.
- Project plans and proposals to determine how well they identify the project purpose, the requirements to address business needs, and the deliverables for each project phase.
- Change management documentation to determine the appropriateness of change decision-making levels and the consistency of approved changes with the project charter and plan.
- Testing and quality management plans to determine the adequacy of quality controls.
- Project closeout documentation to evaluate the entity's ability to inform future projects.

¹²⁴ Refer to U.S. Government Accountability Office's [The Post-Implementation Review](#).